1. Problem Title & Link

Title: LeetCode 875 : Koko Eating Bananas

Link: LeetCode 875 - Koko Eating Bananas

2. Problem Statement (Short Summary)

Koko has several piles of bananas.

piles[i]

represents the number of bananas in each pile.

Koko can eat at a speed of:

k bananas/hour

Rules:

- In one hour, she chooses one pile
- She eats at most k bananas from that pile
- If the pile has fewer bananas, she eats all of them

Find the **minimum eating speed** so Koko can finish within h hours.

3. Examples (Input → Output)

Example 1

Input:

piles=[3,6,7,11]

h=8

Output:

4

Explanation:

At speed = 4:

3 →1 hour

6 →2 hours

7 →2 hours

11→3 hours

Total=8 hours

Example 2

Input:

piles=[30,11,23,4,20]

h=5

Output:

30

Example 3

Input:

piles=[30,11,23,4,20]

h=6

Output:

23

4. Constraints

1 <= piles.length <= 10^4

piles.length <= h <= 10^9

1 <= piles[i] <=10^9

Special restrictions:

- Need minimum valid speed
- Large constraints require optimization

5. Core Concept (Pattern / Topic)

Binary Search on Answer

6. Thought Process (Step-by-Step Explanation)

Brute Force

Try every possible speed:

1,2,3,4,5...

For each speed:

- Calculate required hours
- Return first valid speed

Time:

O(maxPile × n)

Very slow because maximum pile can be:

10^9

Optimized Thinking

Observation:

If speed increases:

Required hours decrease

Example:

Speed=2 → 15 hours

Speed=4 → 8 hours

Speed=10 → 4 hours

This forms a monotonic pattern.

Binary Search can be applied.

Search space:

Minimum speed =1

Maximum speed=max(piles)

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

piles=[3,6,7,11]

h=8

Search:

1 ---- 6 ----11

mid=6

Hours=6

Valid

Search left side

1 ---3---5

mid=3

Hours=10

Invalid

Search right side

4

Answer=4

8. Pseudocode (Language Independent)

```
left=1

right=max(piles)

while left<right:

   mid=(left+right)/2

   hours=0

   for pile in piles:

      hours+=ceil(
            pile/mid
            )

   if hours<=h:

      right=mid

   else:

      left=mid+1

return left
```

9. Code Implementation

Python

```
import math

class Solution:

    def minEatingSpeed(self, piles, h):

        left = 1
        right = max(piles)

        while left < right:

            mid = (left + right) // 2

            hours = 0

            for pile in piles:
                hours += math.ceil(
                    pile / mid
                )

            if hours <= h:
                right = mid
            else:
                left = mid + 1

        return left
```

Java

```
class Solution {

    public int minEatingSpeed(
            int[] piles,
            int h) {

        int left=1;
```

```
        int right=0;

        for(int pile:piles)
            right=Math.max(
                    right,
                    pile
            );

        while(left<right){

            int mid=
                    left+
                    (right-left)/2;

            int hours=0;

            for(int pile:piles){

                hours +=
                        Math.ceil(
                                (double)pile
                                /mid
                        );
            }

            if(hours<=h)
                right=mid;

            else
                left=mid+1;
        }

        return left;
    }
}
```

10. Time & Space Complexity

Time Complexity:

O(n × log(maxPile))

Reason:

- Binary search runs:

log(maxPile)

times

- Each iteration scans all piles

Space Complexity:

O(1)

11. Common Mistakes / Edge Cases

1. Using normal division instead of ceiling

Wrong:

hours += pile/mid

Correct:

hours += ceil(pile/mid)

2. Binary search loop condition mistakes

Wrong:

while(left<=right)

can create extra conditions.

3. Integer overflow

Use:

mid=left+(right-left)/2

4. Missing single pile case

piles=[10]

h=5

12. Detailed Dry Run (Step-by-Step Table)

Input:

piles=[3,6,7,11]

h=8

1	11	6	6	Valid → search left
1	6	3	10	Invalid → search right
4	6	5	8	Valid → search left
4	5	4	8	Valid

Final:

Answer=4

13. Common Use Cases (Real-Life / Interview)

Production scheduling

Server request processing

Bandwidth optimization

Task allocation systems

14. Common Traps (Important!)

1. Forgetting ceiling calculation
2. Wrong binary search updates
3. Integer overflow in mid
4. Brute forcing all speeds

15. Builds To (Related LeetCode Problems)

Progression:

- LC 875 → Koko Eating Bananas
- LC 1011 → Capacity To Ship Packages Within D Days
- LC 410 → Split Array Largest Sum
- LC 1482 → Minimum Days to Make m Bouquets
- LC 1283 → Find Smallest Divisor Given Threshold

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	O(maxPile×n)	O(1)	Too slow
Binary Search on Answer	O(nlog(maxPile))	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

As eating speed increases, required hours always decrease. This monotonic behavior allows binary search to efficiently locate the smallest valid speed.

18. Variations / Follow-Up Questions

1. Return maximum possible speed
2. Multiple workers eating simultaneously
3. Variable eating speed every hour
4. Minimize total cost instead of speed
5. Solve package shipping capacity problem
6.